

EXORCIST: Enabling Atomic-Level Runtime Detection of Spectre Attacks using Precise Event Based Sampling

HAO JIA*, Xidian University, China

HAOYU MA*, Beijing Jiaotong University, China

CHANGFENG DING, Xidian University, China

JINKU LI[†], Xidian University, China

While being key hardware techniques for improving the performance of modern processors, the speculative execution mechanisms also lead to side-channel attacks, which pose significant threats to the security of computer systems. While researchers have proposed various solutions to mitigate the threat posed by speculative attacks, most existing approaches have focused on offline static vulnerability analysis, which suffers from significant limitations of incompleteness and poor scalability.

Based on Intel's Precise Event Based Sampling (PEBS) technique, this paper proposes EXORCIST, a novel runtime framework for detecting Spectre-PHT attacks at the atomic level. Leveraging the key observation that the atomic execution of a Spectre-PHT gadget triggers both a cache miss and a branch mis-precision in a fixed sequence, EXORCIST uses carefully configured PEBS monitoring to efficiently capture all related fine-grained hardware performance events, then launches a kernel- and user-level collaborated taint analysis to effectively pinpoint Spectre-PHT gadgets that have actually exploited vulnerable speculative executions. Our approach significantly reduces the amount of native instructions needed to be processed to confirm actual Spectre exploitation, making it capable of achieving high accuracy and a negligible false positive rate with second-level response time. We have implemented a prototype of EXORCIST and performed a comprehensive evaluation on it. Experimental results indicate that EXORCIST can efficiently detect Spectre-PHT attacks at runtime with acceptable performance overhead, including JIT-compiled Spectre-PHT payloads written in JavaScript that are beyond the reach of existing offline analysis-based approaches.

CCS Concepts: • **Security and privacy** → **Operating systems security; Intrusion detection systems.**

Additional Key Words and Phrases: Spectre attack, Runtime detection

ACM Reference Format:

Hao Jia, Haoyu Ma, Changfeng Ding, and Jinku Li. 2026. EXORCIST: Enabling Atomic-Level Runtime Detection of Spectre Attacks using Precise Event Based Sampling. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE100 (July 2026), 24 pages. <https://doi.org/10.1145/3808107>

1 Introduction

Micro-architectural vulnerability has become an emerging threat to nowadays software and systems security. Among them, speculative execution vulnerabilities [28, 33] exploit key hardware acceleration mechanisms of modern pipelined processors (such as out-of-order and speculative execution),

*Both authors contributed equally to this research.

[†]Corresponding author.

Authors' Contact Information: Hao Jia, Xidian University, Xi'an, China, jiah@stu.xidian.edu.cn; Haoyu Ma, Beijing Jiaotong University, Beijing, China, hyma1@bjtu.edu.cn; Changfeng Ding, Xidian University, Xi'an, China, xueku4ng@163.com; Jinku Li, Xidian University, Xi'an, China, jkli@xidian.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE100

<https://doi.org/10.1145/3808107>

enabling unprivileged processes to violate security boundaries and eavesdrop on sensitive information. Since their 2018 proposal, speculative execution vulnerabilities have shaken the fundamental security logic of existing operating system (OS) designs and undermined other security techniques relying on hardware-based trust and isolation, such as trusted execution environments [9]. Began with two representative examples, namely, Spectre [28] and Meltdown [33], the family of speculative execution vulnerabilities has now grown to many variations [8, 48, 51, 56, 57, 62].

While the operating system security community has coped with earlier attacks such as Meltdown via means like kernel page-table isolation (KPTI) [13], subsequent variants, represented by Spectre-PHT (also known as Spectre-V1), remain valid threats.

A variety of research and engineering efforts have been made to mitigate the threat of speculative attacks. Most existing work focused on detecting and patching vulnerable software or hardware implementations. Although hardware-based solutions aim to solve the problem from the root of cause [1, 24, 25, 34, 45, 50, 61, 63, 65], they are by nature handicapped on the aspect of backward compatibility (i.e., unable to be extended to legacy products due to hardware modification requirement) and could impose a significant burden on manufacturers. Approaches based on software analysis techniques, on the other hand, aim to identify vulnerable code that may be leveraged to launch speculative attacks within a given program [12, 21, 23, 39, 41, 46, 58], yet existing tools of this flavor generally suffer from poor scalability, making them either unable to handle large programs or end up reporting too many false positives [47]. In addition, software-analysis-based speculative vulnerability detection works as a part of the off-line supply chain of software development by design, and thus cannot cope with threats posed by remotely injected attack payloads. Specifically, in scenarios where interpreted execution and JIT (Just-In-Time) compilation are involved (e.g., web applications built with JavaScript), the adversaries could dynamically ship intended speculative attack payload to the victim end, which cannot be vetted by off-line approaches.

An alternative mitigation strategy is to detect speculative attacks in real-time manner. This often involves using machine learning models to analyze micro-architectural statistics from hardware performance counters (HPCs) to identify suspicious speculative execution behaviors at runtime [11, 16, 30, 32, 66]. Intuitively, the effectiveness of these schemes depends on the consistency of statistical distribution between samples used for training and those collected during actual attacks, making them prone to evasive attack strategies in which adversaries reshape the micro-architectural profile of the attack payload to diminish its corresponding behavioral signature and delay its exposure to runtime detectors [31]. Considering speculative attacks (with Spectre-PHT as the example) can leak information at several kilobytes per second [28], lack of promptness would significantly undermine the security value of such statistic-based runtime detectors.

Recognizing these challenges, this paper presents EXORCIST, a novel dynamic speculative attack detection framework capable of identifying variants of Spectre-PHT based on the attack's atomic features. Leveraging Intel's Precise Event Based Sampling (PEBS) [14], an advanced hardware monitoring technique of Intel processors, EXORCIST collects fine-grained hardware performance events, focusing on *cache/TLB misses* and *branch mis-predictions* in particular. Based on the key observation that *an atomic execution of a Spectre-PHT gadget triggers both types of the aforementioned events in a fixed sequence*, our approach then utilizes an ultra light-weight taint analysis module deployed within the operating system kernel to trace data flow dependency between the collected performance events, thus filtering those irrelevant performance events and consequently pinpointing suspicious conditional branches where vulnerable speculative execution is actually triggered. Since PEBS provides a concrete execution context for the sampled events, EXORCIST could thus report these suspected branch locations to a user-level Spectre gadget detection engine, where a second round of static taint analysis is conducted specifically on code snippets targeted by the wrongly speculated control transfers to determine whether potential information leakage has occurred.

With such a cascaded kernel- and user-level collaborative strategy, EXORCIST intervenes only when a particular type of hardware performance event combination is observed. Meanwhile, compared to offline analysis tools, our approach significantly reduces the amount of native instructions needed to be processed to confirm an actual vulnerability exploitation. These together enable EXORCIST to achieve nearly real-time detection of potential Spectre-PHT attacks, which, to the best of our knowledge, has not been accomplished in previous works.

To validate our approach, we have implemented a proof-of-concept prototype of EXORCIST in Linux-x64 on top of an Intel processor with PEBS support, and conducted a series of experiments to evaluate its effectiveness, resilience, and performance. Evaluation results show that, compared to state-of-the-art solutions [21, 23, 46], EXORCIST can effectively detect known Spectre-PHT variants under a variety of compiler optimization settings, surpassing existing solutions on aspects of completeness and scalability. In addition, EXORCIST is capable of dynamically detecting the JIT-compiled Spectre-PHT payload written in JavaScript, which is beyond the reach of any offline approaches. So far, we find that EXORCIST returns no false positives, while being able to respond to the actual Spectre attacks in a matter of seconds even with the presence of statistical evasive strategies.

In general, this paper makes the following contributions:

- We propose a novel approach of using advanced performance monitoring techniques of modern CPUs, represented by Intel PEBS, to capture the essential and atomic features of the execution of Spectre gadgets on-the-fly, which could enable precise and efficient runtime detection of speculative vulnerability exploiting.
- We present a proof-of-concept prototype called EXORCIST based on the above idea, in which we implement a kernel-level taint analysis module specifically for determining data dependency between hardware performance events triggered by Spectre gadgets, as well as a kernel- and user-level collaborated framework to efficiently identify actual Spectre attacks.
- We evaluated EXORCIST with both proof-of-concept Spectre-PHT demos and a JavaScript-based practical attack payload. The results suggest that our approach could achieve near real-time detection of all tested attack variants (even with the present of complex background workloads or evasion attack strategies) with acceptable performance overhead.

To the best of our knowledge, our work is the first to illustrate that by properly utilizing hardware performance counters of modern CPUs, it is possible to achieve practical and accurate runtime detection of transient execution attacks even on nowadays real-world devices running legacy operating systems.

2 Background

Spectre and Speculative Vulnerabilities. Spectre is a family of micro-architectural side-channel attacks which exploit side effects of speculative execution performed by modern processors [22, 28, 29, 36]. Among its variants, Spectre-PHT was the first to be identified. It leverages conditional branch mis-prediction to leak data through cache side channels, as illustrated in Listing 1. Johannesmeyer et al. [23] analyzed the fundamental primitives of the Spectre-PHT gadget, detailing how attackers inject data and use side channels to extract secrets. Such inputs can be supplied through legitimate architectural interfaces or, on SMT-enabled CPUs, across threads using techniques such as load value injection (LVI) [56]. The Spectre-PHT gadget uses various methods to leak secret information, most commonly through cache side channels. Additionally, on SMT-enabled cores, the gadget can leak information through microarchitectural data sampling (MDS) [7] or port contention [6]. AVX2 instructions may also be utilized to leak information [52].

Note that, as mentioned in [28], *rollbacks of the speculative execution mechanism are in fact not always vulnerable*, in that the regular speculative window is too small to run a valid attack payload.

However, in case a conditional branch compares two variables when one of them is not in the data cache, the overhead of a cache miss would then significantly increase the speculative window and consequently allow the execution of full attack payloads. Similarly, TLB misses, executing long instructions such as division, can also achieve this effect. Therefore, *a valid Spectre-PHT attack needs to expand the transient execution window (with one of the aforementioned approaches, e.g., evicting conditional variable `array1_size` of the victim branch in Listing 1 from the cache) before triggering branch mis-prediction*. This key observation leads to the core idea of EXORCIST, i.e., Spectre attacks may be captured with the support of fine-grained hardware performance sampling.

```
if (x < array1_size){
    temp &= array2[array1[x] * 512];
}
```

Listing 1. A basic example of Spectre-PHT payload.

Precise Event Based Sampling. Precise Event Based Sampling (PEBS) is a high-precision hardware sampling mechanism used for performance analysis and security monitoring. It configures the processor’s Performance Monitoring Unit (PMU) to record detailed execution context—such as the instruction pointer, register state, and cache information—when predefined events reach a specified threshold. This threshold, referred to as the *PEBS sampling interval*, balances sampling precision and performance overhead, enabling fine-grained analysis with low runtime cost.

Note that *micro-architectural attacks tend to exploit hardware features that are also considered as performance events*. For Spectre-PHT, cache/TLB misses and branch mis-predictions are performance events that PEBS can sample with high resolution and contextual support. Thus, we leverage PEBS as a core hardware mechanism for Spectre detection.

3 System Design

3.1 Problem Statement & Threat Model

Security Goals. Existing Spectre detection solutions face a series of challenges including backward compatibility, false positives, scalability, capability of identifying injected gadgets, resilience against disguised attacks, etc. To address these challenges, we aim to achieve the following security goals in this work:

- 1) **Prompt and accurate detection.** We want to achieve on-the-fly detection of Spectre attacks, i.e., to respond in near real-time when a Spectre gadget runs and causes information leakage. In addition, as a runtime detection approach, we aim to achieve precise responses.
- 2) **Resilience against evasive strategies.** When defending against Spectre payloads that adopt statistical evasive strategies, we want our approach to remain effective (i.e., it could still achieve nearly real-time and accurate detection).

To further clarify our concept of “prompt and accurate”, we review the known quantitative evidences regarding the volume of information leakage required for a practical attack. Specifically, following the attack methodology adopted in the original Spectre work [28], an attacker in the absence of prior knowledge is limited to random probing within the victim’s address space. Assuming such a scenario (i.e., no prior knowledge of the target address and a uniform distribution of leakage locations), let M denote the size of the victim process’s address space that is subject to side-channel probing, S the size of a sensitive object (e.g., a cryptographic key or session token), and B the total amount of data leaked within a single attack window, the probability that the leaked data overlaps with the sensitive object can be approximated as

$$P_{hit} \approx (S + B)/M \quad \text{for } S + B \ll M$$

To achieve a non-negligible success probability $p_{hit} > p$, the required leakage amount must satisfy

$$B > p \cdot M - S$$

For typical workloads where M is on the order of gigabytes and S is only of dozens of bytes, this inequality implies that B must reach the megabyte scale to make P_{hit} meaningful (e.g., $p = 10^{-3}$). On the other hand, when partial prior knowledge is available—such as searching for the root password hash in kernel memory, where the content is page-aligned and starts with a known prefix (e.g., `root:$`)—the effective search space becomes significantly reduced and structured, allowing a practical attack to be carried out with substantially less leakage required (hundreds of kilobytes in order to pose a realistic security threat) [48, 55]. As such, we argue that *a runtime detection of Spectre attack is prompt and accurate if an attack can be identified and stopped before the caused information leakage exceeds the volume required by a potential attacker with partial prior knowledge.*

Threat Model. As an approach built on top of a hardware mechanism, we assume that the subject system is built with a Linux kernel and runs on a device mounted with an Intel CPU that supports PEBS. The potential adversaries are assumed to be able to launch Spectre-PHT attacks either by leveraging gadgets already existing in the subject system/application, or by generating tailor-made gadgets via JIT compilation of attacker-controlled (source) code. The scenario of injecting hand-crafted binary-level attack payload on-the-fly, on the other hand, is considered as out-of-scope given that an adversary that powerful would be able to achieve information leakage by more straightforward means.

The target of the attack could be a user-level application or from within the kernel space of the subject system. At this point, our work focuses on a class of Spectre-PHT attacks that leak sensitive data via cache-based covert channels. Other Spectre variants, such as Spectre-BTB and Spectre-RSB, as well as Spectre-style attacks that rely on non-cache-based side channels (e.g., port contention or avx channels), are considered out of scope — future work related to these variants are further discussed in Section 6.

Furthermore, it is necessary to emphasize that *the goal of EXORCIST is to identify potential information leakage caused by Spectre-PHT attacks, but NOT the subsequent side-channel measuring operations used to retrieve the leaked data.* This is reasonable because the threat of Spectre-PHT can already be determined once sensitive data is leaked: even without subsequent data retrieving, EXORCIST would still be reporting at least genuine speculative execution vulnerabilities.

3.2 Overview of EXORCIST

Recall that, as emphasized in Section 2, *triggering an exploitable speculative execution for Spectre-PHT would result in the manipulated conditional branch to cause both a cache miss (when loading the condition variable) and a mis-prediction in that exact order, such that the speculative window could be long enough for the subsequent attack payload.* Leveraging this inherent characteristic of the attack, EXORCIST is designed as a 3-leveled screening framework established on top of the advanced hardware performance monitoring technique of PEBS and deployed across kernel and user spaces of the subject system. The high-level idea is:

- With PEBS, EXORCIST looks for paired cache/TLB miss and branch mis-prediction events occurred sequentially within a small time window, and raise a “blue alarm” upon observing such a pairing (potentially vulnerable speculative behavior);
- Next, EXORCIST needs to check the dependency between the paired performance events, and raise a “yellow alarm” if both of them are actually caused by the same conditional branch (vulnerable speculative execution identified, looking for information leakage);

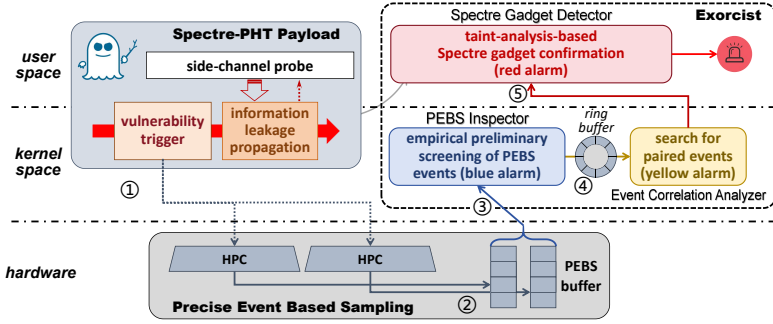


Fig. 1. The Overall Architecture and Workflow of EXORCIST.

- Finally, EXORCIST examines the conditional code snippet executed during the observed branch mis-prediction, and raises a “red alarm” if actual information leakage behaviors can be confirmed (Spectre-PHT detected).

With these 3 levels of screening, our approach could achieve prompt attack detection while avoiding false positives, given that each round of analysis is initiated almost right after the first execution of a vulnerable conditional branch, and will only be confirmed until a “red alarm” is raised.

The overall architecture and workflow of EXORCIST is shown in Figure 1. Specifically, it deploys a *PEBS Inspector* and an *Event Correlation Analyzer* (which is by all means a special-purpose, ultra light-weight taint analysis module) into the OS kernel to handle PEBS events and analyze their dependencies; another taint analysis engine built specifically as the *Spectre Gadget Detector*, on the other hand, is deployed as a user-level service to confirm information leakage behavior of potential Spectre gadgets. When an actual attack is launched, hardware performance counters (HPCs) would pick up both performance events caused by the exploited conditional branch (①) and generate the corresponding event record into a PEBS buffer initialized by EXORCIST (②). The PEBS Inspector of our framework intervenes periodically to collect the buffered records and conduct the aforementioned “blue alarm” preliminary screening based on empirical thresholds with regard to how close a cache miss and a branch mis-prediction be to be considered suspicious (③). A PEBS event pair deemed as potentially risky will then be handed over to the Event Correlation Analyzer to see if both events in it are actually caused by the same conditional branch (④). Once this check raises a “yellow alarm”, that particular PEBS event pair will be further sent to the Spectre Gadget Detector, which, leveraging the contextual information provided in the PEBS records, pinpoints the mis-predicted branch code and distinguishes whether it is a Spectre gadget (⑤). Note that most regular speculative executions would be ignored at ③, while a small portion of them could be handed to ④ and identified as “false alarm”. Only the actually vulnerable speculative executions would be passed to ⑤, where only those actually exploited for information leakage will be eventually confirmed as attacks.

3.3 Empirical Preliminary Screening of PEBS Events

Though the configuration of PEBS is well documented and thus poses no difficulties, certain details in the design of EXORCIST’s PEBS Inspector are still challenging. As the lowest-level and therefore the most performance-sensitive component, the PEBS Inspector needs to be designed delicately to be able to operate timely while not getting invoked too often. Also, the inspection of this module can neither be too relaxed (which would cause too many false negatives) nor too paranoid (which would result in too much intervention of the subsequent modules and therefore cause significant overhead).

Recall that our key observation on Spectre-PHT is that before executing the vulnerable conditional branch, the attack payload triggers a correlated performance event (e.g., a cache miss or a TLB miss), thereby expanding the window for branch mis-prediction. This indicates that the paired performance events due to an attack behavior would likely be correlated temporally. As such, EXORCIST configures PEBS to particularly capture these types of events, together with their timestamps, addresses of source instructions, as well as the general register values when these events occurred. Considering that the amount of cache misses caused by the overall operations of a system is intuitively much larger than that of the branch mis-prediction, and also given that PEBS uses a dedicated buffer to store its records according to the temporal sequence, our PEBS Inspector is let to investigate the buffered events in the backward order while using branch mis-prediction as its “beacon” events. This means that, starting from the last record within the PEBS buffer, the PEBS Inspector is let to look for the most recent branch mis-prediction event, then switch to search for a cache miss event or TLB miss event which happened shortly before the focused branch mis-prediction. This secondary search terminates when all cache/TLB miss events within a pre-determined set of temporal/positional thresholds have been ruled out, or when another branch mis-prediction event occurs before a match is found. Since PEBS is designed to record 1 out of n continuous events (with n being the sampling interval), rare cases, where irrelevant branch mis-predictions happen to occur between events triggering an attack, are unlikely to affect the screening process since the unrelated event may not be captured.

In all cases, the PEBS Inspector switches back to look for the next “beacon” until all records within the PEBS buffer are examined. Note that PEBS raises a software interruption when its event buffer is about to overflow, causing mandatory intervention of the user-specified event handler (in our case, the PEBS Inspector). To avoid the overhead of raising additional interrupts, EXORCIST deploys its PEBS Inspector as an extension of the Linux kernel’s CPU scheduling routine, letting it intervene during the time slice switching between threads. The advantage of this design is in two folds: it reuses interrupts raised by the operating system’s scheduler rather than causing extra ones, thus avoiding further slowing down the subject system; also, since the PEBS Inspector expires the PEBS buffer upon each investigation, having it operate between time slices allows EXORCIST to easily determine the origin of those suspicious PEBS event pairs it discovers (i.e., the exact thread that occupies the previous time slice).

The rule used to identify suspicious pairs of PEBS events involves a heuristic notion of “shortly before.” We studied and analyzed the known implementations of Spectre-PHT variants provided by Kocher [27], and found that *in a naturally compiled attack payload*, the execution time between the instruction triggering a cache miss and the one causing a branch mis-prediction typically does not exceed 200 CPU cycles. To be conservative, we set a looser empirical threshold for EXORCIST’s PEBS checker: a cache miss event followed by a branch mis-prediction is considered potentially suspicious only if they occur within a 300-cycle window.

3.4 Taint-Based Event Correlation Analysis

Note that by no means can the PEBS records reflect a semantical relationship between the captured performance events. This means that when the PEBS Inspector reports a suspicious pair of cache miss and branch mis-prediction events according to its empirical rules, it cannot assert that both of them are caused by the execution of the same conditional branching operation. Therefore, EXORCIST leverages static taint analysis to determine whether an event pair reported by the PEBS Inspector is indeed semantically dependent.

Constructing the Event Correlation Analyzer as a kernel-level taint analysis module, and in doing so balancing effectiveness and efficiency, is non-trivial. To begin with (and as a necessary preparation), binary instructions between those that caused the suspected cache miss and branch

mis-prediction events are retrieved and disassembled using a linear scan algorithm. A typical challenge of disassembling in-memory binaries of x86 or x64, which is a Complex Instruction Set Computer (CISC) architecture, is the variable-length instruction encoding that usually increases the difficulty of determining the start of each instruction. Fortunately, EXORCIST avoids this issue, as PEBS captures addresses corresponding to the start of instructions that triggered the events, providing an accurate disassembly initiation point.

Next, the Event Correlation Analyzer of EXORCIST proceeds to carry out an ultra light-weight taint analysis on the disassembled binary snippet mentioned above. Specifically, the analysis starts from the instruction that triggered the suspected cache/TLB miss, which could be either a data transfer, an arithmetic operation, or a logical operation instruction in which values are read from the memory. In the initial taint setup, all registers, flags, and memory addresses affected by this particular instruction are marked as tainted. Since PEBS provides a concrete execution state of the captured events, the Event Correlation Analyzer thus leverages the known register values of the tainted instruction to simulate the execution of the disassembled snippet, while employing symbolic representation for other unknown values. During the simulation process, the Event Correlation Analyzer needs to explore all potential execution paths, while limiting the total number of instructions executed within each path. Meanwhile, the Event Correlation Analyzer conducts taint propagation according to a quite regular set of rules, namely:

- If the source operand of a data transfer instruction is tainted, then the destination operand will be tainted.
- If any source operand of an arithmetic/logical instruction is tainted, the destination register and the affected flags are also tainted.
- A memory barrier instruction prevents taint propagation.

Eventually, if the Event Correlation Analyzer finds the instruction triggering the suspected branch mis-prediction depends on a tainted value, it concludes that there is a clear dependency between the two suspected events, i.e., these events are caused by the same condition branching behavior.

```

1      mov    0x2e8e(%rip),%ecx           # cache miss
2      movslq %ecx,%rcx
3      mov    -0x8(%rbp),%rax
4      cmp    %rcx,%rax                  # %rax and -0x8(%rbp) tainted
5      jae    11f9 <victim_function+0x49> # branch mis-prediction
6      mov    -0x8(%rbp),%rcx            # %rcx tainted
7      lea    0x2e8b(%rip),%rax          # %rax cleared
8      movzbl (%rax,%rcx,1),%eax         # %eax tainted
9      shl    $0x9,%eax                  # %eax tainted
10     movslq %eax,%rcx                  # %rcx tainted
11     lea    0x2f1a(%rip),%rax          # %eax cleared
12     movzbl (%rax,%rcx,1),%ecx         # detection

```

Listing 2. Taint Propagation in a Proof-of-Concept Spectre-PHT BCB Gadget.

Listing 2 demonstrates the assembly representation of a vanilla example of a conditional branch exploited by Spectre-PHT, in which data transfer instruction at line 1 triggers a cache miss when reading a pointer-referenced value. As such, the Event Correlation Analyzer of EXORCIST would mark both the source and destination operands (a pointer symbol and the ecx register) of this instruction as tainted. Subsequently, the instruction at line 2 would propagate the taint from ecx to the rcx register, while the next instruction at line 4 further propagates the taint to the carry flag cf (according to how the instruction cmp is supposed to work). Therefore, when the Event Correlation Analyzer finds that the last instruction at line 5 is jae, which not only triggers branch

mis-prediction, but also has its behavior governed by the value of `cf`, it will determine that the two performance events triggered at lines 1 and 5 are semantically dependent.

3.5 Taint-Based Spectre Gadget Confirmation

A vulnerable speculative execution does not lead to a Spectre-PHT attack without transferring control to a Spectre gadget after the mis-predicted branching. Therefore, EXORCIST must confirm the existence of such a gadget for each pair of PEBS events (or, in other words, each vulnerably executed conditional branch) reported by the Event Correlation Analyzer. With the general register values after executing the cache-miss-triggering memory loading during the vulnerable conditional branching provided by PEBS, flag values that govern the mis-predicted control transfer can be computed, allowing EXORCIST to understand exactly which piece of code was executed during the speculative execution, and hence perform a pinpointed taint analysis with the assistance of concrete runtime context.

Initial Taint Setting. EXORCIST's Spectre Gadget Detector relies highly on the validity of initial taints. Recall that the branch instruction that triggers the mis-prediction event relies on the value of a specific (set of) flag(s). Changes to the flag(s) are usually caused by either a `cmp` or a `test` instruction. In case of a `cmp` instruction that compares two variables, having known that one of them originates from the suspected cache-missed memory loading, EXORCIST simply marks the other one as the initial taint. In all other cases (e.g., a `test` instruction or a `cmp` comparing a variable with an immediate value), EXORCIST starts the taint analysis from the mis-predicted branch instruction, and mark all subsequent register values that are retrieved from the memory as initial taints.

Spectre-PHT can be mounted with several types of gadgets, which include *Bounds Check Bypass* (BCB), *Bounds Check Bypass Store* (BCBS) [47], and *Speculative Type Confusion* [26]. A BCBS gadget simply exploits the vulnerable speculative execution to modify arbitrary (out-of-bound) memory locations, making it easy to identify. A BCB gadget, on the other hand, involves leaking a secret from an out-of-bound memory location first, then launching another secret-indexed memory loading to leave a measurable cache line trace. This allows it to be implemented into a variety of mutations. Additionally, type confusion is a new type of attack vector that exploits multiple branch mis-predictions to cause the victim to execute with variables holding values of the wrong type, thereby leaking memory contents.

Handling BCB. The core logic of EXORCIST's gadget identification routine with regard to the detection of BCB-style Spectre-PHT gadgets complies with a similar model as proposed in [47]. The process involves three levels of tainting. Those registers that participated in the vulnerable branching operation and might contain value from untrusted sources (which are labeled during the kernel-level analysis of the Event Correlation Analyzer) are initially tainted (henceforth referred to as *taint1*), and the taint level can be raised during the subsequent propagation if the following three types of operations are identified:

- **Secret acquisition:** A memory location indexed by a *taint1* value is accessed (including both read and write operations), indicating potential information leakage. This will raise the taint level of the so-accessed location to *taint2*.
- **Value amplification:** A *taint2* value is amplified, for example, by multiplication. BCB gadgets do this to map the leaked secret to a larger address space, allowing safe recover via cache side-channel probing without interference from hardware features like prefetching. Note that only when a *taint2* value gets amplified would EXORCIST raise it to *taint3*.
- **Value used as an index:** When a *taint3* value is used as an array index, EXORCIST can then safely assume that a covert channel for information leakage has been established, and will therefore report a Spectre-PHT attack.

Note that in most cases, Spectre-PHT only leaks a small value (typically one byte) at a time when adopting cache-based side channels. Hence, in such scenarios, EXORCIST only promotes taint for memory accesses of less than 16 bits. In addition, a BCB gadget may be mutated to leak the secret via control flow, as shown in listing 3. Therefore, if a value marked as *taint2* influences a conditional branch and causes distinguishable memory accesses, EXORCIST also reports detection of a gadget.

```

if (x < array1_size) {
    if (array1[x] == k)
        temp &= array2[0];
}

```

Listing 3. BCB Variant That Leaks Information Via Control Flow.

Here we again use Listing 2 to illustrate the taint propagation procedure of EXORCIST's Spectre Gadget Detector. The branch misprediction is triggered by a *jae* (line 5), governed by a *cmp* between *rax* and *rcx* (line 4), where *rcx* originates from a cache-missed load (line 1). Thus, EXORCIST marks *rax* as *taint1*. In addition, having found that the value of *rax* is retrieved from the memory location `-0x8(%rbp)` at line 3, the in-memory value at this particular location is also marked as *taint1*. Therefore, when the tainted in-memory value is retrieved to register *rcx* subsequently, it causes the *taint1* mark to be propagated (line 7). Since *rcx* was then immediately used as part of the index to retrieve another in-memory value (an out-of-bound secret), this causes the destination register value at *eax* to be raised to *taint2* (Line 9). The *eax* value was then amplified at line 10, causing its mark to be further raised to *taint3* and then propagated to *rcx* due to a register value transmission (line 11). Eventually, this *taint3*-marked *rcx* was used as part of the index in a memory access (line 13), at which point EXORCIST will report that a BCB gadget for Spectre-PHT attack is detected.

Handling Type Confusion. The major difference between type confusion and BCB gadgets is that the information leakage is achieved with multiple branch mis-predictions carried out sequentially. Though in this case, EXORCIST reports correlation only between the cache/TLB miss event and the first branch mis-prediction event, the detection of type confusion can still be carried out in a way similar to the BCB detection approach. Specifically, during the step-by-step execution of instructions, we calculate the outcomes of subsequent branch instructions based on the values in the registers. If the result cannot be determined through calculation or if the branch depends on values related to triggering cache miss instructions, then both targets of the branch will be explored, allowing the aforementioned taint rules to propagate on Spectre-PHT gadgets that exploit type confusion.

4 Implementation

We implemented a proof-of-concept prototype for EXORCIST (targeting Intel 12th Gen Core processors), which consists of a kernel module (with about 7.5K lines of C code) that handles PEBS inspection and event correlation analysis, as well as a user-level application (with about 29K lines of C++ code) implementing the Spectre gadget identifier. Data transmissions between the two components are done through netlink and shared memory, with the former used for passing control information and the latter employed for transferring data such as PEBS records and binary machine code.

4.1 PEBS Sampling

As a hardware feature, PEBS works at the level of individual processor cores, thus EXORCIST needs to initiate its PEBS configuration and maintain an independent PEBS buffer for each CPU core respectively. Note that the targeted processor features a heterogeneous architecture with P-cores and E-cores, and PEBS does not support cache miss sampling on E-cores. Therefore, EXORCIST instead uses the `MEM_LOAD_UOPS_RETIRED.DRAM_HIT` event to approximate memory access latency on E-cores.

Since time slice switching is a frequently called kernel routine, minimizing the execution time of EXORCIST's inserted code is crucial to reduce overhead. To achieve this, we allocate a contiguous memory block as a ring buffer between the PEBS Inspector and Event Correlation Analyzer (Figure 1) to store suspicious PEBS records from preliminary screening. We also implement deduplication before placing data in the buffer, reducing memory writes and preventing excessive overhead. This ensures efficient data collection with low latency during CPU scheduling.

4.2 Taint Analysis Modules

Though both modules are based on taint analysis, the kernel-level Event Correlation Analyzer and the user-level Spectre Gadget Detector differ significantly in their implementations. Specifically, the Event Correlation Analyzer continuously reads data from the previously described ring buffer. It employs the Beaengine disassembly engine [5], which is capable of running within the Linux kernel, to disassemble binary instructions. In addition, the taint analysis engine used by this module was developed in-house. Once the analysis results in a "yellow alarm," the Event Correlation Analyzer transmits the corresponding PEBS event pairs along with other related data into a shared memory region for further analysis.

The Spectre Gadget Detector communicates control information with the kernel modules of EXORCIST via Netlink to start and stop the detection process. Once detection begins, the Spectre Gadget Detector maintains two processes to respectively listen to the Netlink channel and the shared memory channel, while another four processes are created to concurrently analyze cases reported by the Event Correlation Analyzer. Note that when a detection procedure reaches the Spectre Gadget Detector, more code fragments need to be simulated. When encountering branch instructions targeting addresses outside of the retrieved code fragments, EXORCIST cannot simply discard them, thus additional code fragments must be retrieved. Therefore, the Spectre gadget detector retrieves the additional required binary instructions from the operating system kernel.

5 Evaluation

We evaluate EXORCIST in terms of detection effectiveness, deployability, and performance overhead. Detection effectiveness is validated using Spectre-PHT proof-of-concept attacks in both native and JavaScript settings [3, 27]. Deployability is assessed under diverse benign workloads, including representative application scenarios, SPEC CPU2017 [54], and server-side applications such as Nginx and MySQL. Performance overhead is measured using CPU- and memory-intensive workloads, including SPEC CPU2017, Graph500 [40], Nginx, MySQL, and synthetic stress benchmarks.

All experiments were conducted on an Ubuntu 22.04 LTS system with an Intel 12th Gen Core i7-12700 processor. To ensure the soundness, we verified whether Kocher's Spectre-PHT variants could successfully leak information under different compiler optimizations. We compiled the attack demos using Clang 11.1.0 and GCC 11.4.0 with optimization levels `-O0`, `-O1`, and `-O2`. The resulting binaries were then assessed to determine whether information leakage was indeed exhibited. In most cases, the compiled demos successfully leaked data, except for a few exceptions:

First, Listing 4 shows the assembly code of attack variant v6, compiled with GCC at the `-O2` optimization level. In this code segment, `rax` holds the array size used to constrain memory accesses, while `rdi` contains a potentially out-of-bounds value. The `and` instruction on the second line applies a bitmask to `rdi` and stores the sanitized result into `rax`. Later, the memory access (e.g., line 9) uses `rax`, which now contains a safe value, thereby preventing any information leakage.

Second, for variant v8, all optimization levels above `-O0` introduced the `cmov` instruction—a conditional move available on modern x86 processors that transfers data between registers without modifying the control flow. As a result, speculative execution was suppressed, and the attack failed.

Finally, variant v13 involves an inline function check. Under certain compilation conditions—specifically, GCC at $-O0$ and Clang at both $-O0$ and $-O1$ —the function’s behavior was not preserved, resulting in an incomplete payload. Consequently, this variant was excluded from our evaluation.

```

1          # if ((x & array1_size) == x)
2      movslq  array1_size(%rip), %rax
3      andq    %rdi, %rax
4      cmpq    %rdi, %rax
5      je      .L5
6      ret
7      .L5:  # temp &= array2[array1[x] * 512];
8      leaq    array1(%rip), %rdx
9      movzbl  (%rdx,%rax), %eax
10     leaq    array2(%rip), %rdx
11     sall    $9, %eax
12     cltq
13     movzbl  (%rdx,%rax), %eax

```

Listing 4. Partial Assembly Code of Spectre-PHT v6.

Definition of False Negative Detections. Based on our security goal as claimed in Section 3.1, we further categorize the outcomes of EXORCIST’s detection into three types.

- 1) **Detected in time.** A detection is considered timely if the attack is detected before causing a significant volume of information leakage (exceeding the requirement of a potential attacker with partial prior knowledge). To be more rigorous, we set this threshold of leakage to 100 KB, which is below the reported leakage volume according to existing work [48, 55].
- 2) **Detected behind time.** Following the above definition, a detection is classified as untimely if achieved after the detected attack has successfully caused a leakage exceeding the aforementioned threshold, indicating a practical security risk.
- 3) **Not detected.** Finally, an attack is regarded as undetected if the attacker successfully leaks sensitive data without triggering any detection.

Naturally, both “Detected behind time” and “Not detected” are considered as false negatives, as the attack is not prevented promptly and accurately as required by our security goal.

5.1 Effectiveness

In this experiment, all of Kocher’s Spectre-PHT variants (except for the excluded v13 instances) were run on a Linux system with EXORCIST deployed. For comparison, binaries of all the tested instances were also analyzed with a series of state-of-the-art off-line Spectre vulnerability analysis tools. Selected benchmarks include SPECTECTOR and CAT, a pair of static analysis tools based on symbolic execution and axiomatic bounded model checking, as well as KASPER, a dynamic taint-based analyzer. To the best of our knowledge, these are the latest works reported on the topic.

Two major takeaways are obtained from the result shown in Table 1. First, *EXORCIST exhibited better effectiveness than the compared benchmarks when the scalability of detection tools comes into question*. Specifically, we found that CAT does not support position-independent code (PIC) and has incomplete support for assembly instructions. As a result, we had to manually modify the attack instances being tested, including replacing position-independent addresses with addressing modes that do not rely on RIP, and converting unsupported assembly instructions into equivalent instructions supported by CAT, in order to “adapt the analyzer”. Similarly, KASPER only supports binaries compiled with Clang, and even then, it fails to produce correct results under $-O2$ optimization.

Table 1. Effectiveness Comparison of Different Spectre Analysis Tools (EXORCIST, CAT, SPECTECTOR, and KASPER) on Kocher’s Spectre-PHT Variants.

tool	compiler & optimization level	Spectre-PHT variants														
		v1	v2	v3	v4	v5	v6	v7	v8	v9	v10	v11	v12	v13	v14	v15
EXORCIST	clang-O0/O1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	✓	✓
	clang-O2	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	gcc-O0	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	✓	✓
	gcc-O1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	gcc-O2	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
CAT/ SPECTECTOR	clang-O0/O1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	✓	✓
	clang-O2	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	gcc-O0	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	✓	✓
	gcc-O1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	gcc-O2	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓	✓	✓	✓
KASPER	clang-O0/O1	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	✓	✓
	CLANG-O2	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?
	gcc-O0	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–
	gcc-O1	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–
	gcc-O2	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–

“✓”: accurate results; “✗”: false positives/negatives; “–”: cases omitted; “?”: unable to produce results.

Second, by leveraging runtime context, EXORCIST achieved higher accuracy in identifying ambiguous cases. For example, both SPECTECTOR and CAT falsely reported the compiler-thwarted instance in Listing 4 as a valid Spectre-PHT attack, while EXORCIST *correctly avoided this false positive*. Runtime analysis showed that EXORCIST precisely marked `rdi` as the initial taint, and since the actual array index was stored in `rdx`, no taint propagation occurred. These results demonstrate that with PEBS support, EXORCIST outperforms offline Spectre detection tools in handling such edge cases.

Case Study 1: Detecting Injected Attack Payload. To further evaluate EXORCIST’s ability to detect Spectre attacks launched via remote payload injection, we conducted a second experiment using a JavaScript-based Spectre-PHT implementation [3] executed in the Google Chrome. Listing 5 shows the assembly code reported by EXORCIST during JIT compilation of the payload, with comments indicating the corresponding JS code for clarity. EXORCIST detected the attack immediately after this code was executed and correctly attributed it to the browser’s address space. This highlights EXORCIST’s advantage over offline tools, which are unable to handle payloads injected at runtime.

Case Study 2: Co-Existing with Benign Payloads. As a runtime attack detector, it is essential to evaluate the false positive rate of EXORCIST, since false alarms may misclassify benign processes as malicious and potentially terminate legitimate tasks. By design, EXORCIST raises alarms only when a code fragment exhibits concrete leakage semantics; functionalities that are semantically incapable of leaking sensitive data is not reported, thereby avoiding false positives. This case study further examines whether EXORCIST raises any red alarms during benign executions with a series of tests and real-world applications. These include SPEC 2017 benchmarks, as well as server-side applications such as Nginx and MySQL, following the same setup as the performance evaluation in Section 5.4. In addition, following [43], we include seven representative scenarios (iozone on both local disks and Network File System (NFS), fio [4] on NFS, glmark2 [20], iperf [19], OpenSSL [42], and lmbench [38]), and, following [32], three interactive workloads (web browsing with Firefox, video streaming via browser plugins, and file operations in a text editor). Across all these experiments, *our approach produces no red alarms*. This is reasonable because although previous researches suggested that a real-world program may contain a significant amount of

Spectre gadgets [47], regular executions would not cause such vulnerable code to act in the way as in actual Spectre attacks and consequently activate the first-stage alarms of EXORCIST; on the other hand, should a Spectre gadget of such kind causes data leakage during a benign execution, it would indicate that a potentially exploitable vulnerability, making its detection meaningful.

```

1          # if ((index|0) < (arr_size|0)){
2      movzx  edx, byte ptr [rbx + rdx]
3      cmp    eax, edx
4      jge    0xef30a9bea2d
5          # index = simpleByteArray[index|0]|0;
6      mov    eax, eax          # the detection code
7      cmp    rax, rcx          # inserted by the
8      jae    0xef30a9bea3a     # browser
9      movzx  eax, byte ptr [rbx + rax]
10     mov    edx, dword ptr [rsi]
11     # index = (index << 12)|0;
12     sub    edx, 1
13     shl    eax, 0xc
14     # index = (index & ((TABLE1_BYTES-1)|0))|0;
15     and    edx, eax
16     # junk = (junk ^ (probeTable[index]|0))|0;
17     mov    eax, dword ptr [rsi + 8]
18     cmp    rdx, rcx
19     jae    0xef30a9bea3e
20     movzx  edi, byte ptr [rbx + rdx]

```

Listing 5. Binary Assembly of JavaScript Spectre-PHT Payload (After JIT compiling) Captured by Exorcist.

To further highlight the significance of EXORCIST's 3-leveled screening strategy leveraging taint tracking in suppressing false positives, we conducted 3-minute benign test runs on MySQL and Nginx, respectively, during which 27,374/26,827 blue alarms and 1,117/1,683 yellow alarms were observed while no red alarm was reported. Even when monitoring the subject system under a relatively idle condition, we could still observe 1,549 blue alarms and 40 yellow alarms raised by EXORCIST within approximately 3 minutes, but again, no red alarm was reported. These indicate that the two taint-tracking-based components of EXORCIST eliminated potential false alarms effectively.

Case Study 3: Detection Capability Under Complex Environments. Exorcist operates on a time-slice basis, precisely associating sampled PEBS events with their corresponding processes. The collected event sequences capture the atomic micro-architectural signatures of Spectre attacks, namely the brief cache and branch mis-prediction effects induced by probing a single byte. Embedding attack payloads into different workloads does not alter these signatures or their occurrence within the victim's time slices. Once such a signature is detected, it is forwarded to the taint-tracking module, which is likewise agnostic to workload complexity. Once detected, such signatures are forwarded to the taint-tracking module, which is agnostic to workload complexity. Consequently, EXORCIST consistently detects attacks regardless of whether they run in isolation or within complex workloads. To evaluate this, we construct three representative scenarios:

- First, a synthetic high-load benchmark that launches multiple worker threads generating intensive cache misses and branch mis-predictions, while a separate thread concurrently executes a Spectre-PHT attack.
- Second, an Nginx-based performance benchmark to emulate realistic background server workloads, with the Spectre-PHT attack launched in parallel.

- And finally, we combine the aforementioned background workloads, and run the Spectre-PHT attack alongside both, forming a mixed and more challenging environment.

Experimental results show that EXORCIST can consistently detect the Spectre-PHT attack in a timely manner across all three scenarios, even when the attack is intentionally slowed down by up to $7\times$ using sleep-based delays, and under sampling rates of both 50 and 100.

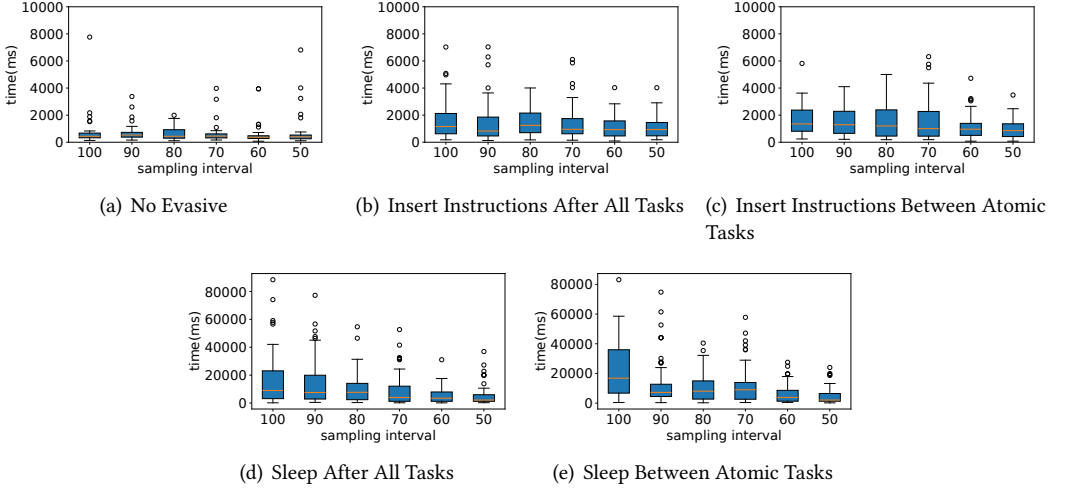


Fig. 2. The Time Required to Detect the Attack Under Different Evasion Strategies.

5.2 Resilience Against Evasive Strategies

In a previous work, Li et al. [31] divided the Spectre-PHT attack process into three atomic tasks: flushing the cache line, leaking information to the cache line, and guessing the information from the cache. He then proposed four evasive attack strategies, namely 1) put the attack to sleep in between atomic tasks; 2) put the attack to sleep after all tasks have been completed; 3) insert instructions in between atomic tasks; and 4) insert instructions after all tasks have been completed. All these strategies could significantly alter the statistical signature of the resulting attack payloads, making them harder to discover by analyzing hardware performance statistics.

To evaluate EXORCIST's effectiveness against evasive attacks, we built four Spectre-PHT attack demos, each implementing one of the aforementioned evasive strategies. For each demonstration, we executed it 50 times with EXORCIST enabled and recorded the time required to detect the attack, as shown in Figure 2. All these demos were tested under different sampling intervals with a sevenfold slowdown of the attack. The results indicate that evasion strategies based on instruction insertion have little impact on the detection effectiveness of EXORCIST, whereas the two sleep-based evasion strategies have a more pronounced effect. Specifically, attacks using the two instruction-insertion evasion methods are detected within 8 seconds in most cases, with an average detection time below 2.5 seconds. By contrast, attacks employing the two sleep-based evasion methods are typically detected within 80 seconds, with an average below 25 seconds. So far, the worst individual case we observed was an attempt of such evasive attack eventually detected after 89 seconds. Intuitively, sleep causes frequent context switches, which in turn affects the collection of event pairs in the first stage of EXORCIST and consequently resulted in longer lifetime of the disguised attacks.

Table 2. Comparison Between EXORCIST and Machine-Learning-Based State-of-The-Arts

	False positives	Test case coverage		Effectiveness under evasion strategies
		Attack samples	Benign samples	
EXORCIST	×	15 Spectre-PHT variants	SPEC 2017, MySQL, Nginx	Effective (longer time of detection)
Pan et al.[44]	✓	Not specified	Not specified	Degraded (accuracy ↓)
Li et al.[32]	✓	1 (unspecified) Spectre-PHT variant	Firefox, (unspecified) text editors, (customized) memory- and CPU-intensive workloads	Degraded (accuracy ↓)
Oshana et al.[43]	✓	Vanilla Spectre-PHT	9 (unspecified) applications	N/A
Wang et al.[60]	✓	1 (unspecified) Spectre-PHT variant	GNU Binutils, SPEC 2006	Degraded (accuracy ↓)

The above results demonstrate the strong effectiveness of EXORCIST against evasion strategies: all disguised attacks are detected within a reasonable time window, significantly limiting potential leakage. For example, while Spectre-PHT attacks without evasion can leak data at 2 KB/S [32], the presence of EXORCIST constrains sleep-based evasion attacks to at most 25 KB of leakage (roughly equivalent to 12–13 seconds of a full attack). As this leakage remains well below the attacker threat threshold defined in Section 3.1, all such cases are considered timely detections, with insufficient data exposure to cause practical damage.

5.3 Comparison with State-of-The-Arts

To the best of our knowledge, most (if not all) previous runtime detection methods against Spectre are machine-learning-based techniques using statistical performance indicators [32, 43, 44, 60].

However, none of these approaches are properly reproducible, preventing a quantitative comparison with EXORCIST. Therefore, we provide a theoretical and qualitative analysis. Table 2 compares EXORCIST with state-of-the-art ML-based runtime detectors. In general, existing approaches suffer from some (or all) of the following limitations:

Non-Negligible False Positives. Machine learning is by nature probabilistic, so false positive is an inevitable concern. Specific to the real-world scenario of Spectre detection, false positive would cause unacceptable consequences such as wrongfully terminating benign tasks, causing unnecessary blockings, etc. In contrast, EXORCIST avoids false positives with a conservative model-based multilevel screening, which only responds when all key factors of a Spectre-PHT data leakage have been confirmed. In fact, as a runtime detector, EXORCIST is designed to tolerate false negatives, in that missing the leakage of an individual byte would not hurt as long as the attack is deterministically caught before any meaningful information has been revealed.

Insufficient Evidence on Effectiveness. Specifically, detection models used by the existing machine-learning-based approaches are typically trained with attack samples extracted from merely a limited subset of Spectre-PHT variants (usually focusing on the most typical forms); similarly, the benign execution samples used to set up baselines for these models are also constructed in an overly simple way (e.g., taken from nothing else but the execution of SPEC benchmarks). Such configurations of model training make the effectiveness of these approaches questionable when considering the capacity of being generalized to more complex scenarios. In contrast, EXORCIST has been validated on 15 known Spectre-PHT variants, and its soundness when running with a wide range of benign payloads (including real-world applications) has also been verified.

Prone to Evasion Strategies. So far, existing machine-learning-based Spectre detectors either reported significant performance degradation when facing attacks that adopt evasion strategies,

or did not assess the effectiveness against evasion techniques at all. In contrast, EXORCIST has demonstrated strong resilience against typical evasion strategies (see Section 5.2).

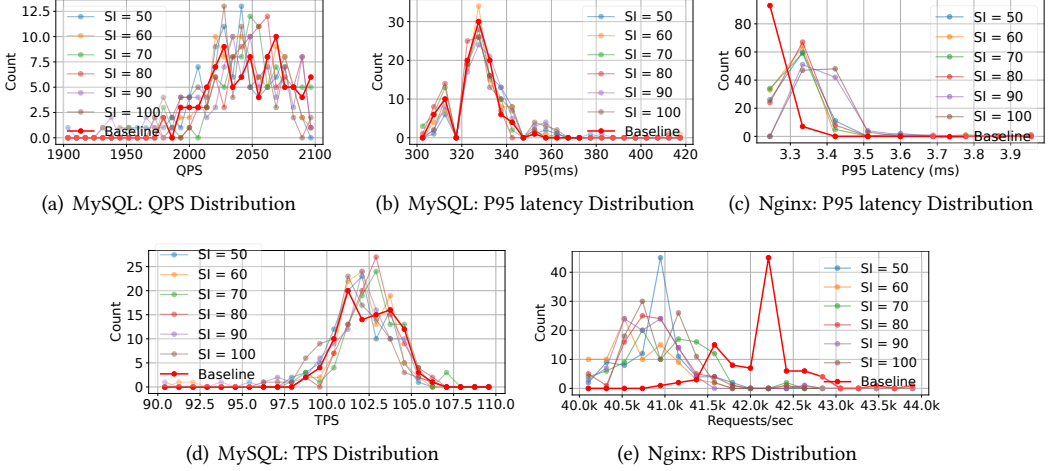


Fig. 3. The Performance Impact of EXORCIST on MySQL and Nginx. (“SI” Indicates Sampling Interval)

5.4 Performance

Representative Real-World Workloads. To evaluate the performance impact of EXORCIST in real-world scenarios, we consider two representative server types: a database server and a web server, using MySQL and Nginx as test subjects. For MySQL, we employ Sysbench’s `oltp_read_write` workload over 10 tables with 100,000 records each, using 20 threads for 120 seconds. To mitigate cache effects, the system idles for 60 seconds between runs after a sync operation. For Nginx, we use `wrk` to generate client workloads with 100 connections, while adopting the same 20-thread configuration, 120-second duration, and cache mitigation strategy as in the MySQL evaluation.

100 samples of TPS (transactions/sec), QPS (queries/sec), and P95 latency (the value below which 95% of the observed response time fall) were collected for MySQL under each condition (i.e., without EXORCIST, and with EXORCIST running at the sampling interval of 50, 60, 70, 80, 90, and 100). For Nginx, on the other hand, throughput (represented by Requests/sec, RPS) and P95 latency were sampled instead. As shown in Figure 3, compared to the baseline results, the performance degradation caused by EXORCIST was insignificant for both tested servers. The collected MySQL QPS/TPS and P95 show almost no difference compared to the baseline, while the Nginx RPS distribution shifts slightly to the left with a decrease of about 2.4%, and its P95 shifts slightly to the right with an increase of about 1.5%. These results indicate that EXORCIST has only a negligible impact on both MySQL and Nginx.

Memory and CPU Stressing Workloads. Next, we evaluate the performance overhead introduced by EXORCIST under memory- and CPU-stressing workloads, which are designed to induce frequent cache misses and branch mis-predictions. Such workloads represent worst-case background noise for our detection mechanism.

We first use the `SpecSpeed2017_int_base` benchmarks from the SPEC 2017 suite as a representative CPU-intensive workload to establish a baseline. As shown in Fig. 4, setting the sampling interval to 50 reduces the overall score by an average of 1.15 points. As the interval increases from 60 to 100, the overall scores drop by 1.04, 0.93, 0.85, 0.82, and 0.77 points, respectively.

Table 3. Performance Impact Under Stress Workloads

Workload	Sampling Interval (SI)						
	Baseline	50	60	70	80	90	100
Graph500 (TEPS, 10^8)	2.87	2.37↓ 17.4%	2.44↓ 15.0%	2.47↓ 13.9%	2.50↓ 12.9%	2.51↓ 12.5%	2.53↓ 11.8%
Worst-case (Time, s)	16.29	17.17↑ 5.4%	17.09↑ 4.9%	17.01↑ 4.4%	17.07↑ 4.8%	16.73↑ 2.7%	16.75↑ 2.8%

In addition, we evaluate EXORCIST under two stress workloads designed to aggressively exercise memory and branch behavior. The first is the Graph500 benchmark, which features highly irregular memory accesses. We use the MPI reference implementation with 20 threads on a graph of 2^{24} vertices and an average degree of 32, reporting the harmonic mean of traversed edges per second (TEPS) for breadth-first search (BFS) traversals, under sampling intervals from 50 to 100. The second is a synthetic worst-case workload that allocates a 512 MB memory region and performs random memory accesses followed by data-dependent branches, inducing frequent cache misses and branch mis-predictions. The workload traverses the memory region once, and we report the average execution time over 20 runs under the same sampling intervals. We report the performance overhead of EXORCIST under the two stress workloads in Table 3. Under the most aggressive sampling interval, the BFS throughput in Graph500 decreases by up to 17.4%, while the average execution time of the worst-case workload increases by up to 5.4%. This suggests that the performance impact of EXORCIST is acceptable even in the scenario of stress workloads.

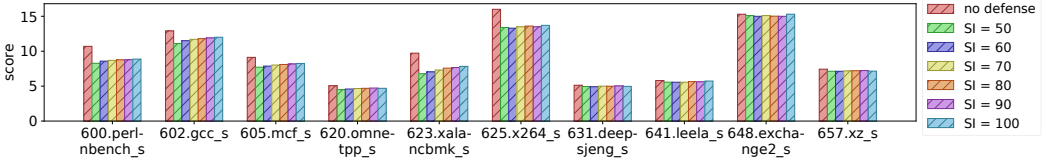


Fig. 4. Impact of EXORCIST on CPU Performance (“SI” Indicates Sampling Interval)

Context Switch Overhead. Since EXORCIST introduces additional code to the context switch routine, its impact on the OS kernel’s context switch performance was evaluated, too. Though tools like lmbench [38] are commonly used for such measurements, they assume minimal computation per process and thus would be over-optimistic for reflecting realistic overhead. As such, the previously used real-world workloads (MySQL, Nginx and SPEC 2017) were once again leveraged. Specifically, the two servers were allowed to run for one hour each, during which the time spent of EXORCIST additional code in context switch phases was recorded. The distributions of the measured values given in Fig. 5 show that in the majority of cases, overhead induced by EXORCIST fell within 100 nanoseconds, 75% of which are less than 80, 32, 17 nanoseconds for MySQL, Nginx and SPEC 2017, respectively. For reference, lmbench reports an average context switch time of about 680 nanoseconds without EXORCIST, indicating that our overhead is acceptable.

Cost of Static Analysis. With the two static analysis components, EXORCIST involves runtime simulation of instruction executions at both kernel and user spaces. These components simulate only very small code snippets. Specifically, the kernel-level Event Correlation Analyzer executes an average of 6.12 instructions, taking 30.308 ms, whereas the user-level Spectre Gadget Detector executes an average of 57.43 instructions with 79.38 ms. Moreover, the de-duplication mechanism (see Section 4.1) would further reduce the intervention frequency of these components in that once a code snippet is considered clear, it would not be simulated again subsequently.

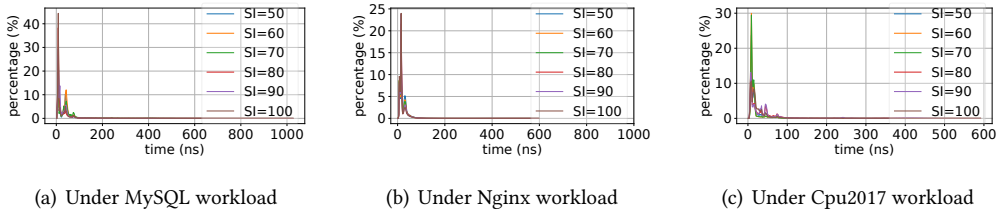


Fig. 5. Increase in Process Switch Latency (Clipped to 99%).

6 Discussion

Although PEBS is a powerful performance analysis mechanism that provides precise event sampling, it has some inaccuracies and limitations, which might introduce a potential impact on our approach. For instance, Yi et al. found that PEBS suffers from the shadow effect—a time delay between the PMU counter overflow and the arming of the PEBS hardware, during which any event that occurs cannot be captured [64]. However, this window is very small, and we are searching across a large number of events. Therefore, we believe that it is unlikely to significantly impact our work, which has been demonstrated by our evaluation results. The PEBS sampling interval is governed by low-level hardware mechanisms. In multi-core systems with multi-threaded scheduling, this makes it extremely difficult for user-level attackers to precisely align attack execution to consistently evade sampling. Nevertheless, the sampling interval directly affects both detection coverage and runtime overhead. Exploring adaptive sampling strategies to achieve a better security–performance tradeoff may indeed improve the effectiveness of EXORCIST. We leave this to the future work.

Regarding backward compatibility, EXORCIST requires PEBS support, which has been available since the Intel Skylake micro-architecture released in 2015. This enables our approach to be deployed on a wide range of existing systems. In addition to Intel’s PEBS, other hardware platforms like AMD’s IBS [17] and ARM’s SPE [2] also support precise sampling. This potentially allows our approach to be extended to the mentioned processors with minor adaptations (as long as high-precision monitoring of cache-miss and branch mis-prediction events is supported).

Note that using static analysis in EXORCIST presents challenges as it operates on assembly code fragments in the Spectre Gadget Detector. During simulated execution, conditional or branch instructions may arise. If the target address is fixed, it can be retrieved from memory; otherwise, techniques like hooks are needed. However, these may impact performance and response time, warranting careful consideration and leaving it for future work.

Currently, integrating EXORCIST requires modifying certain source code of the OS kernel, which might, to a certain extent, limit its upstream in the industrial community. Techniques like kprobes [37] and emerging methods like eBPF [18] can mount probes without modifying the kernel, which will be adopted to improve the design of EXORCIST in future work.

Recall that long-latency instructions (e.g., division) can potentially be exploited to extend the transient execution window in Spectre-PHT attacks. Since these cannot be directly captured by PEBS, a complementary detection strategy is to identify regressive branch executions and mis-predictions triggered by the same branch instruction. This idea is motivated by the observation that Spectre-PHT relies on training the victim branch predictor before triggering the leakage. We plan to further investigate this approach in future work.

Extending EXORCIST to other Spectre variants (e.g., Spectre-BTB or Spectre-RSB) is feasible in principle, as the PEBS-based detection paradigm remains applicable. However, these variants rely on

indirect control-flow transfers rather than conditional branches, requiring architectural extensions to recover transient execution targets. Moreover, EXORCIST currently focuses on cache-based covert channels. Extending it to non-cache channels (e.g., port contention or AVX-based channels) would require capturing and correlating additional classes of microarchitectural signals. We leave these extensions and the required architectural support as future work.

7 Related Work

Speculative Payload Analysis. Existing methods for analyzing Spectre gadgets use both static and dynamic analysis tools. Earlier static tools model syntax patterns to search for specific gadget patterns in code [12, 39], but could often generate false positives and are limited by their pattern specificity. More recent work aims to detect Spectre gadgets in more general ways, and adopts advanced software analysis techniques such as symbolic execution, taint analysis, semantics-based model checking, and deep learning to achieve more precise modeling of the vulnerabilities [21, 35, 46, 58]. There are also tools [53, 67] that aim to produce programs without speculative vulnerabilities by implementing secure compilers. From another perspective, dynamic-style approaches mainly leverage data flow analysis with runtime contexts to simulate speculative execution scenarios and therefrom detect Spectre gadgets in both user-space programs and the Linux kernel [23, 41, 47].

Runtime Mitigation. Current online detection of speculative attacks falls into two main categories, i.e., machine learning-based detection and rule-based detection. Machine learning-based approaches were originally proposed as attack-oriented methods [15, 16, 66], which work by gathering runtime information of attack programs and use it to train classifiers to detect attacks. Victim-oriented approaches, e.g., anomaly-based detection [10], were proposed more recently, which leverage HPC sampling data from benign programs to train classifiers and thus eliminate the need for attack samples. However, obtaining data from only a single source can lead to high false-positive rates, and identified attacks cannot be further classified. Attackers may even attempt to evade detection by slowing down the attack or mimicking benign programs [32]. Meanwhile, rule-based detection solutions adopted a learning-free approach by manually extracting cache set access patterns from known attack programs to establish detection strategies [49, 59]. However, existing approaches either targeted the side channel probing phase (rather than the speculative information leakage) of the attack, or overly relied on known PoC attack demos, thus limiting their effectiveness.

8 Conclusion

In this paper, we present EXORCIST, a practical runtime system for detecting Spectre-PHT attacks using Intel's Precise Event-Based Sampling (PEBS) and lightweight taint analysis. Our evaluation across multiple Spectre-PHT variants, including attacks embedded in complex workloads, shows that EXORCIST achieves accurate, near real-time detection with acceptable performance overhead, provided that the platform offers sufficiently precise micro-architectural event sampling. More broadly, our work demonstrates that precise hardware event sampling can serve as an effective foundation for practical runtime defenses against transient execution attacks.

Data Availability

The tool and data are available at <https://github.com/corvus10086/EXORCIST>.

Acknowledgments

The authors would like to thank all reviewers for their valuable comments. This work was partially supported by the Beijing-Tianjin-Hebei Natural Science Cooperation Project (No. 25JJJC0005), the Talent Fund of Beijing Jiaotong University (No. 2025XKRC013), the Key Research and Development Program of Shaanxi (Program No. 2025CY-YBXM-066), and the '111 Center' (B16037).

References

- [1] Sam Ainsworth and Timothy M. Jones. 2020. MuonTrap: preventing cross-domain spectre-like attacks by capturing speculative state. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (Virtual Event) (ISCA '20)*. IEEE Press, 132–144. doi:10.1109/ISCA45697.2020.00022
- [2] ARM. 2024. ARM Documentation: Sampling and Profiling Extension. <https://developer.arm.com/documentation/109429/latest/>
- [3] ascendr. 2018. Spectre Chrome. <https://github.com/ascendr/spectre-chrome>
- [4] Jens Axboe. 2025. fio: Flexible I/O Tester. <https://github.com/axboe/fio>
- [5] BeaEngine. 2024. BeaEngine: A Powerful x86/x64 Disassembler and Decompiler. <https://github.com/BeaEngine/beaengine>
- [6] Atri Bhattacharyya, Alexandra Sandulescu, Matthias Neugschwandtner, Alessandro Sorniotti, Babak Falsafi, Mathias Payer, and Anil Kurmus. 2019. SMOtherSpectre: Exploiting Speculative Execution through Port Contention. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (London, United Kingdom) (CCS '19)*. Association for Computing Machinery, New York, NY, USA, 785–800. doi:10.1145/3319535.3363194
- [7] Claudio Canella, Daniel Genkin, Lukas Giner, Daniel Gruss, Moritz Lipp, Marina Minkin, Daniel Moghimi, Frank Piessens, Michael Schwarz, Berk Sunar, Jo Van Bulck, and Yuval Yarom. 2019. Fallout: Leaking Data on Meltdown-resistant CPUs. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (London, United Kingdom) (CCS '19)*. Association for Computing Machinery, New York, NY, USA, 769–784. doi:10.1145/3319535.3363219
- [8] Claudio Canella, Jo Van Bulck, Michael Schwarz, Moritz Lipp, Benjamin Von Berg, Philipp Ortner, Frank Piessens, Dmitry Evtvushkin, and Daniel Gruss. 2019. A systematic evaluation of transient execution attacks and defenses. In *Proceedings of the 28th USENIX Conference on Security Symposium (Santa Clara, CA, USA) (SEC'19)*. USENIX Association, USA, 249–266.
- [9] Guoxing Chen, Sanchuan Chen, Yuan Xiao, Yinqian Zhang, Zhiqiang Lin, and Ten H. Lai. 2019. SgxPectre: Stealing Intel Secret Keys from SGX Enclaves Via Speculative Execution. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. 142–157. doi:10.1109/EuroSP.2019.00020
- [10] Marco Chiappetta, Erkan Savas, and Cemal Yilmaz. 2016. Real time detection of cache-based side-channel attacks using hardware performance counters. *Appl. Soft Comput.* 49, C (Dec. 2016), 1162–1174. doi:10.1016/j.asoc.2016.09.014
- [11] JongHyeon Cho, TaeHyun Kim, TaeHun Kim, and Youngjoo Shin. 2019. Real-Time Detection on Cache Side Channel Attacks using Performance Counter Monitor. In *2019 International Conference on Information and Communication Technology Convergence (ICTC)*. 175–177. doi:10.1109/ICTC46691.2019.8939797
- [12] Nick Clifton. 2018. SPECTRE Variant 1 scanning tool. <https://access.redhat.com/blogs/766093/posts/3510331>
- [13] Jonathan Corbet. 2017. The current state of kernel page-table isolation. <https://lwn.net/Articles/741878>
- [14] Intel Corporation. 2024. Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3B. System Programming Guide.
- [15] John Demme, Matthew Maycock, Jared Schmitz, Adrian Tang, Adam Waksman, Simha Sethumadhavan, and Salvatore Stolfo. 2013. On the feasibility of online malware detection with performance counters. *SIGARCH Comput. Archit. News* 41, 3 (June 2013), 559–570. doi:10.1145/2508148.2485970
- [16] Jonas Depoix and Philipp Altmeyer. 2018. Detecting Spectre Attacks by identifying Cache Side-Channel Attacks using Machine Learning. *Betriebssysteme.Org* (2018). Issue August.
- [17] Paul J. Drongowski. 2007. *Instruction-Based Sampling: A New Performance Analysis Technique for AMD Family 10h Processors*. Technical Report. AMD. https://www.amd.com/content/dam/amd/en/documents/archived-tech-docs/white-papers/AMD_IBS_paper_EN.pdf
- [18] eBPF Project. 2024. eBPF. <https://ebpf.io/>
- [19] Energy Sciences Network (ESnet). 2025. iperf: TCP/UDP Bandwidth Measurement Tool. <https://iperf.fr>
- [20] glmark2 Developers. 2025. glmark2: OpenGL Benchmark. <https://github.com/glmark2/glmark2>
- [21] Marco Guarnieri, Boris Köpf, José F. Morales, Jan Reineke, and Andrés Sánchez. 2020. Spectector: Principled Detection of Speculative Information Flows. In *2020 IEEE Symposium on Security and Privacy (SP)*. 1–19. doi:10.1109/SP40000.2020.00011
- [22] J. Horn. 2018. Speculative execution, variant 4: Speculative store bypass. <https://bugs.chromium.org/p/project-zero/issues/detail?id=1528>
- [23] Brian Johannesmeyer, Jakob Koschel, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2022. KASPER: Scanning for Generalized Transient Execution Gadgets in the Linux Kernel. In *29th Annual Network and Distributed System Security Symposium, NDSS 2022*. doi:10.14722/ndss.2022.24221
- [24] Khaled N. Khasawneh, Esmail Mohammadian Koruyeh, Chengyu Song, Dmitry Evtvushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. 2019. SafeSpec: Banishing the Spectre of a Meltdown with Leakage-Free Speculation. In *Proceedings of the 56th Annual Design Automation Conference 2019 (Las Vegas, NV, USA) (DAC '19)*. Association for

- Computing Machinery, New York, NY, USA, Article 60, 6 pages. doi:10.1145/3316781.3317903
- [25] Vladimir Kiriansky, Ilia Lebedev, Saman Amarasinghe, Srinivas Devadas, and Joel Emer. 2018. DAWG: a defense against cache timing attacks in speculative execution processors. In *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture (Fukuoka, Japan) (MICRO-51)*. IEEE Press, 974–987. doi:10.1109/MICRO.2018.00083
 - [26] Ofek Kirzner and Adam Morrison. 2021. An Analysis of Speculative Type Confusion Vulnerabilities in the Wild. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2399–2416. <https://www.usenix.org/conference/usenixsecurity21/presentation/kirzner>
 - [27] Paul Kocher. 2018. Microsoft Compiler Spectre Mitigation. <https://www.paulkocher.com/doc/MicrosoftCompilerSpectreMitigation.html>
 - [28] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. 2020. Spectre attacks: exploiting speculative execution. *Commun. ACM* 63, 7 (June 2020), 93–101. doi:10.1145/3399742
 - [29] Esmaeil Mohammadian Koryueh, Khaled N. Khasawneh, Chengyu Song, and Nael Abu-Ghazaleh. 2024. Spectre Returns! Speculation Attacks Using the Return Stack Buffer. *IEEE Design & Test* 41, 2 (2024), 47–55. doi:10.1109/MDAT.2024.3352537
 - [30] Congmiao Li and Jean-Luc Gaudiot. 2019. Detecting Malicious Attacks Exploiting Hardware Vulnerabilities Using Performance Counters. In *2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*, Vol. 1. 588–597. doi:10.1109/COMPSAC.2019.00090
 - [31] Congmiao Li and Jean-Luc Gaudiot. 2020. Challenges in Detecting an “Evasive Spectre”. *IEEE Computer Architecture Letters* 19, 1 (2020), 18–21. doi:10.1109/LCA.2020.2976069
 - [32] Congmiao Li and Jean-Luc Gaudiot. 2022. Detecting Spectre Attacks Using Hardware Performance Counters. *IEEE Trans. Comput.* 71, 6 (2022), 1320–1331. doi:10.1109/TC.2021.3082471
 - [33] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, Mike Hamburg, and Raoul Strackx. 2020. Meltdown: reading kernel memory from user space. *Commun. ACM* 63, 6 (May 2020), 46–56. doi:10.1145/3357033
 - [34] Kevin Loughlin, Ian Neal, Jiacheng Ma, Elisa Tsai, Ofir Weisse, Satish Narayanasamy, and Baris Kasikci. 2021. DOLMA: Securing Speculation with the Principle of Transient Non-Observability. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 1397–1414. <https://www.usenix.org/conference/usenixsecurity21/presentation/loughlin>
 - [35] Chuan Lu, Senlin Luo, and Limin Pan. 2025. SCR-Spectre: Spectre gadget detection method with strengthened context relevance. *Comput. Electr. Eng.* 123, PA (April 2025), 16 pages. doi:10.1016/j.compeleceng.2024.110029
 - [36] Giorgi Maisuradze and Christian Rossow. 2018. ret2spec: Speculative Execution Using Return Stack Buffers. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (Toronto, Canada) (CCS '18)*. Association for Computing Machinery, New York, NY, USA, 2109–2122. doi:10.1145/3243734.3243761
 - [37] Ananth Mavinakayanahalli, Prasanna Panchamukhi, Jim Keniston, Anil Keshavamurthy, and Masami Hiramatsu. 2006. Probing the guts of kprobes. In *Linux Symposium*, Vol. 6. 5.
 - [38] Larry McVoy and Carl Staelin. 1996. lmbench: Portable Tools for Performance Analysis. In *USENIX 1996 Annual Technical Conference (USENIX ATC 96)*. USENIX Association, San Diego, CA. <https://www.usenix.org/conference/usenix-1996-annual-technical-conference/lmbench-portable-tools-performance-analysis>
 - [39] Microsoft. 2019. MSVC compiler reference: /Qspectre. <https://docs.microsoft.com/en-us/cpp/build/reference/qspectre?view=vs-2019>
 - [40] Richard C. Murphy, Kyle B. Wheeler, Brian W. Barrett, and James A. Ang. 2010. Introducing the Graph 500. In *Cray Users Group (CUG)*. 45–74.
 - [41] Oleksii Oleksenko, Bohdan Trach, Mark Silberstein, and Christof Fetzter. 2020. SpecFuzz: bringing spectre-type vulnerabilities to the surface. In *Proceedings of the 29th USENIX Conference on Security Symposium (SEC'20)*. USENIX Association, USA, Article 84, 18 pages.
 - [42] OpenSSL Software Foundation. 2025. OpenSSL: The Open Source Toolkit for SSL/TLS. <https://www.openssl.org>
 - [43] Rob Oshana, Mitchell A. Thornton, Eric C. Larson, and Xavier Roumégue. 2021. Real-Time Edge Processing Detection of Malicious Attacks Using Machine Learning and Processor Core Events. In *2021 IEEE International Systems Conference (SysCon)*. 1–8. doi:10.1109/SysCon48628.2021.9447078
 - [44] Zhixin Pan and Prabhat Mishra. 2021. Automated Detection of Spectre and Meltdown Attacks Using Explainable Machine Learning. In *2021 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. 24–34. doi:10.1109/HOST49136.2021.9702278
 - [45] Arash Pashrashid, Ali Hajiabadi, and Trevor E. Carlson. 2023. HidFix: Efficient Mitigation of Cache-Based Spectre Attacks Through Hidden Rollbacks. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–9. doi:10.1109/ICCAD57390.2023.10323979

- [46] Hernán Ponce-de Leon and Johannes Kinder. 2022. Cats vs. Spectre: An Axiomatic Approach to Modeling Speculative Execution Attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*. 235–248. doi:10.1109/SP46214.2022.9833774
- [47] Zhenxiao Qi, Qian Feng, Yueqiang Cheng, Mengjia Yan, Peng Li, Heng Yin, and Tao Wei. 2021. SpecTaint: Speculative Taint Analysis for Discovering Spectre Gadgets. In *28th Annual Network and Distributed System Security Symposium, NDSS 2021*. doi:10.14722/ndss.2021.24466
- [48] Hany Ragab, Andrea Mambretti, Anil Kurmus, and Cristiano Giuffrida. 2024. GhostRace: exploiting and mitigating speculative race conditions. In *Proceedings of the 33rd USENIX Conference on Security Symposium* (Philadelphia, PA, USA) (SEC '24). USENIX Association, USA, Article 346, 18 pages.
- [49] Majid Sabbagh, Yungsi Fei, Thomas Wahl, and A. Adam Ding. 2018. SCADET: a side-channel attack detection tool for tracking prime+probe. In *Proceedings of the International Conference on Computer-Aided Design* (San Diego, California) (ICCAD '18). Association for Computing Machinery, New York, NY, USA, Article 107, 8 pages. doi:10.1145/3240765.3240844
- [50] Gururaj Saileshwar and Moinuddin K. Qureshi. 2019. CleanupSpec: An "Undo" Approach to Safe Speculation. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (MICRO-52). Association for Computing Machinery, New York, NY, USA, 73–86. doi:10.1145/3352460.3358314
- [51] Michael Schwarz, Moritz Lipp, Daniel Moghimi, Jo Van Bulck, Julian Stecklina, Thomas Prescher, and Daniel Gruss. 2019. ZombieLoad: Cross-Privilege-Boundary Data Sampling. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (London, United Kingdom) (CCS '19). Association for Computing Machinery, New York, NY, USA, 753–768. doi:10.1145/3319535.3354252
- [52] Michael Schwarz, Martin Schwarzl, Moritz Lipp, Jon Masters, and Daniel Gruss. 2019. NetSpectre: Read Arbitrary Memory over Network. In *Computer Security – ESORICS 2019: 24th European Symposium on Research in Computer Security, Luxembourg, September 23–27, 2019, Proceedings, Part I* (Luxembourg, Luxembourg). Springer-Verlag, Berlin, Heidelberg, 279–299. doi:10.1007/978-3-030-29959-0_14
- [53] Basavesh Ammanaghatta Shivakumar, Gilles Barthe, Benjamin Gregoire, Vincent Laporte, Tiago Oliveira, Swarn Priya, Peter Schwabe, and Lucas Tabary-Maujean. 2023. Typing High-Speed Cryptography against Spectre v1. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1094–1111. doi:10.1109/SP46215.2023.10179418
- [54] Standard Performance Evaluation Corporation. 2017. SPEC CPU2017 Benchmark Suite. <https://www.spec.org/cpu2017/>
- [55] Daniël Trujillo, Johannes Wikner, and Kaveh Razavi. 2023. INCEPTION: exposing new attack surfaces with training in transient execution. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) (SEC '23). USENIX Association, USA, Article 409, 18 pages.
- [56] Jo Van Bulck, Daniel Moghimi, Michael Schwarz, Moritz Lippi, Marina Minkin, Daniel Genkin, Yuval Yarom, Berk Sunar, Daniel Gruss, and Frank Piessens. 2020. LVI: Hijacking Transient Execution through Microarchitectural Load Value Injection. In *2020 IEEE Symposium on Security and Privacy (SP)*. 54–72. doi:10.1109/SP40000.2020.00089
- [57] Stephan van Schaik, Alyssa Milburn, Sebastian Österlund, Pietro Frigo, Giorgi Maisuradze, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2019. RIDL: Rogue In-Flight Data Load. In *2019 IEEE Symposium on Security and Privacy (SP)*. 88–105. doi:10.1109/SP.2019.00087
- [58] Guanhua Wang, Sudipta Chattopadhyay, Ivan Gotovchits, Tulika Mitra, and Abhik Roychoudhury. 2021. oo7: Low-Overhead Defense Against Spectre Attacks via Program Analysis. *IEEE Transactions on Software Engineering* 47, 11 (2021), 2504–2519. doi:10.1109/TSE.2019.2953709
- [59] Limin Wang, Lei Bu, and Fu Song. 2023. SCAGuard: Detection and Classification of Cache Side-Channel Attacks via Attack Behavior Modeling and Similarity Comparison. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. doi:10.1109/DAC56929.2023.10247890
- [60] Wubing Wang, Guoxing Chen, Yueqiang Cheng, Yinqian Zhang, and Zhiqiang Lin. 2021. Specularizer: Detecting Speculative Execution Attacks via Performance Tracing. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 18th International Conference, DIMVA 2021, Virtual Event, July 14–16, 2021, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 151–172. doi:10.1007/978-3-030-80825-9_8
- [61] Ofir Weisse, Ian Neal, Kevin Loughlin, Thomas F. Wenisch, and Baris Kasikci. 2019. NDA: Preventing Speculative Execution Attacks at Their Source. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (MICRO-52). Association for Computing Machinery, New York, NY, USA, 572–586. doi:10.1145/3352460.3358306
- [62] Johannes Wikner and Kaveh Razavi. 2022. {RETBLEED}: Arbitrary Speculative Code Execution with Return Instructions. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 3825–3842. <https://www.usenix.org/conference/usenixsecurity22/presentation/wikner>
- [63] Mengjia Yan, Jiho Choi, Dimitrios Skarlatos, Adam Morrison, Christopher W. Fletcher, and Josep Torrellas. 2018. InvisiSpec: making speculative execution invisible in the cache hierarchy. In *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture* (Fukuoka, Japan) (MICRO-51). IEEE Press, 428–441. doi:10.1109/MICRO.2018.00042

- [64] Jifei Yi, Benchao Dong, Mingkai Dong, and Haibo Chen. 2020. On the precision of precise event based sampling. In *Proceedings of the 11th ACM SIGOPS Asia-Pacific Workshop on Systems* (Tsukuba, Japan) (*APSys '20*). Association for Computing Machinery, New York, NY, USA, 98–105. doi:[10.1145/3409963.3410490](https://doi.org/10.1145/3409963.3410490)
- [65] Jiyong Yu, Mengjia Yan, Artem Khyzha, Adam Morrison, Josep Torrellas, and Christopher W. Fletcher. 2019. Speculative Taint Tracking (STT): A Comprehensive Protection for Speculatively Accessed Data. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (*MICRO-52*). Association for Computing Machinery, New York, NY, USA, 954–968. doi:[10.1145/3352460.3358274](https://doi.org/10.1145/3352460.3358274)
- [66] Yunjie Zhang and Yiorgos Makris. 2020. Hardware-Based Detection of Spectre Attacks: A Machine Learning Approach. In *2020 Asian Hardware Oriented Security and Trust Symposium (AsianHOST)*. 1–6. doi:[10.1109/AsianHOST51057.2020.9358255](https://doi.org/10.1109/AsianHOST51057.2020.9358255)
- [67] Zhiyuan Zhang, Gilles Barthe, Chitchanok Chuengsatiansup, Peter Schwabe, and Yuval Yarom. 2023. Ultimate SLH: taking speculative load hardening to the next level. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) (*SEC '23*). USENIX Association, USA, Article 399, 18 pages.

Received 2026-02-13; accepted 2026-03-24